SOURCE CODE / ARCHITECTURE / GIT HISTORY

企业舆情风险智能体平台

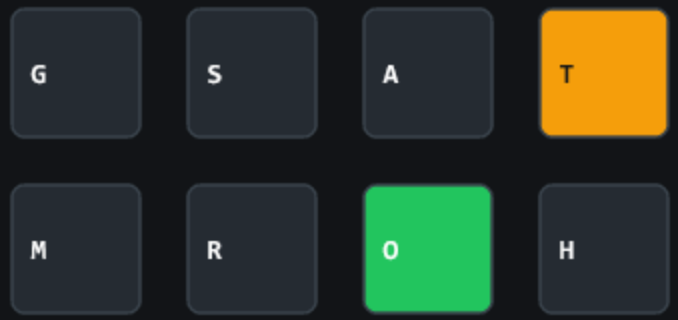
源码与架构学习手册

从一次请求出发，理解 LangGraph、Multi-Agent、Memory、MCP、RAG 与 Observability

---

代码基线 origin/main @ 12e2f0e

46 slides · 2026-09-22



阅读基线与事实边界

默认主分支

origin/main

12e2f0e

当前工作树

```
HEAD      41f180b
branch    codex/retire-legacy-package
diff      HEAD..origin/main = empty
```

状态标签

IMPLEMENTED — 生产路径可达，代码与测试均有证据

OPTIONAL — 实现存在，但需要配置或外部基础设施

NOT WIRED — 仅测试、文档或残留命名，不在生产默认图

git branch -a -vv; git diff HEAD origin/main; langgraph.json

系统解决的业务问题

企业面对的不是“搜到多少网页”，而是能否把公开信号与内部事实对齐，并形成可审计的风险判断和应对方案。



核心约束：区分事实、指控、传言、解释和建议；保留来源与不确定性。

运行入口与技术栈

```
LangGraph API / Studio
langgraph.json → deep_researcher()

FastAPI Web
POST /api/research → graph.astream()

Console scripts
deep-research-web
social-media-mcp
apify-social-api
```

Orchestration	LangGraph ≥ 0.5.4
Agent runtime	LangChain Core 1.2.x
Models	OpenAI / Anthropic / DeepSeek / Google / Groq / AWS
Retrieval	Tavily + native search + local RAG
Storage	Milvus / FAISS / Chroma / MySQL / Neo4j
Protocol	MCP adapters + MCP 1.9+
Web	FastAPI + SSE
Trace	LangSmith

pyproject.toml; langgraph.json; src/open_deep_research/web/server.py

当前目录地图

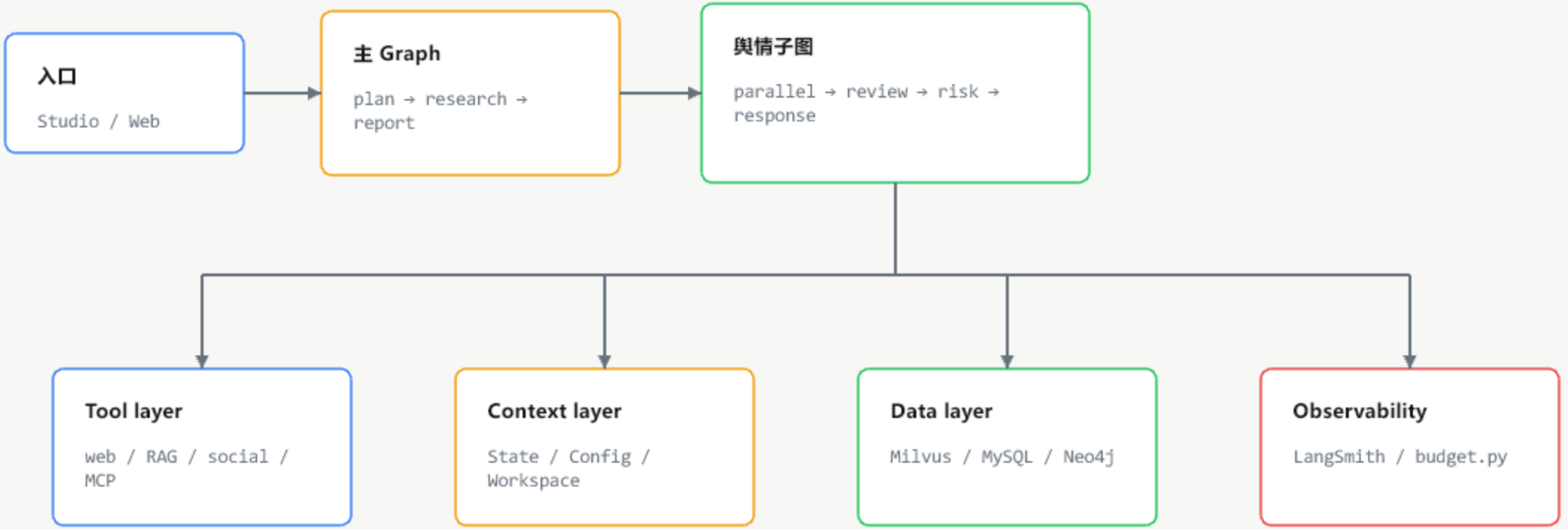
```
src/open_deep_research/
├─ deep_researcher.py    # 主图与舆情子图
├─ state.py              # 五域 State + reducers
├─ runtime/              # 共享 AgentRuntime
├─ public_opinion_agents/ # 四个角色合同
├─ research_graph/       # 每次 run 的证据图与工作区
├─ rag/                  # 知识库索引与检索
├─ memory/               # MySQL 长期记忆
├─ mcp/                  # 多 Server 工具接入
├─ observability/        # LangSmith 边界
├─ social_media/         # 进程内社媒工具
└─ web/                  # FastAPI + 静态前端
```

- 先读 deep_researcher.py：图是什么
- 再读 state.py：数据怎样合并
- 接着读 runtime/：Agent 怎样循环
- 按问题进入 mcp/、rag/、memory/
- README 的 skills/ 目录当前不存在

目录名只是线索；PPT 中的结论都回到调用关系与测试验证。

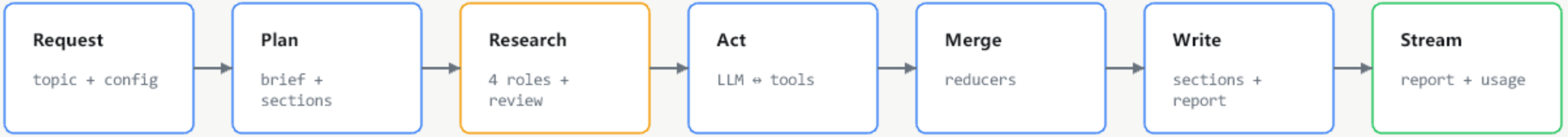
rg --files src/open_deep_research; README.md

当前总体架构



可选能力：RAG、MCP、Research Graph、MySQL Memory、LangSmith 均由配置启用。

一次请求的鸟瞰流程



每一步都能映射到函数和 State patch；没有隐藏的“自动 AgentExecutor”。

```
HumanMessage → DeepResearchState → AgentSpec → tool_calls → ToolMessage → reducers → report.final
```

web/server.py:research; deep_researcher.py; runtime/agent_runtime.py

项目全貌学习检查点

看完这一章，我应该能回答

01 默认主分支、当前 HEAD 和真实入口分别是什么？

02 主 Graph 与舆情子图如何嵌套？

03 哪些基础设施默认启用，哪些必须显式配置？

04 目录中哪个模块负责流程，哪个模块负责 Agent 循环？

05 系统的业务产出为什么不等同于搜索摘要？

Slides 2–7; source index in docs/project-learning/sources.md

两条外部调用入口

LangGraph API / Studio

```
graphs["Deep Researcher"]
  = deep_researcher.py:deep_researcher


graph = factory(config)
await graph.ainvoke(input, config)
```

FastAPI SSE

```
POST /api/research
  → create thread_id
  → config["configurable"]
  → graph.astream(..., "updates")
  → SSE status / stream / report / usage
```

两条入口最终都调用同一个原生 Pregel；Web 入口额外负责校验、并发限制、SSE 和使用量回传。

主 Graph 的创建与编译

```
builder = StateGraph(
  DeepResearchState,
  input=AgentInputState,
  config_schema=Configuration,
)

return builder.compile(
  name="public_opinion_research"
)
```

- 模块加载时保留 deep_researcher_builder，供测试和工具使用
- 导出的 deep_researcher 是 factory，每次调用重新 compile
- config 参数不直接注入 factory；节点从 RunnableConfig 读取
- 生产 compile 没有传 checkpointer 或 store

可打断点： `_create_deep_researcher_builder() → deep_researcher()`

deep_researcher.py:_create_deep_researcher_builder;deep_researcher

State 的五个所有权域

messages

LangGraph native

对话与最终 AIMessage

workflow

brief / round / review / tasks

流程控制

agents

role report / memory / summary

按角色合并

research

run_id / working_contexts

研究图上下文

report

sections / completed / final

写作产物

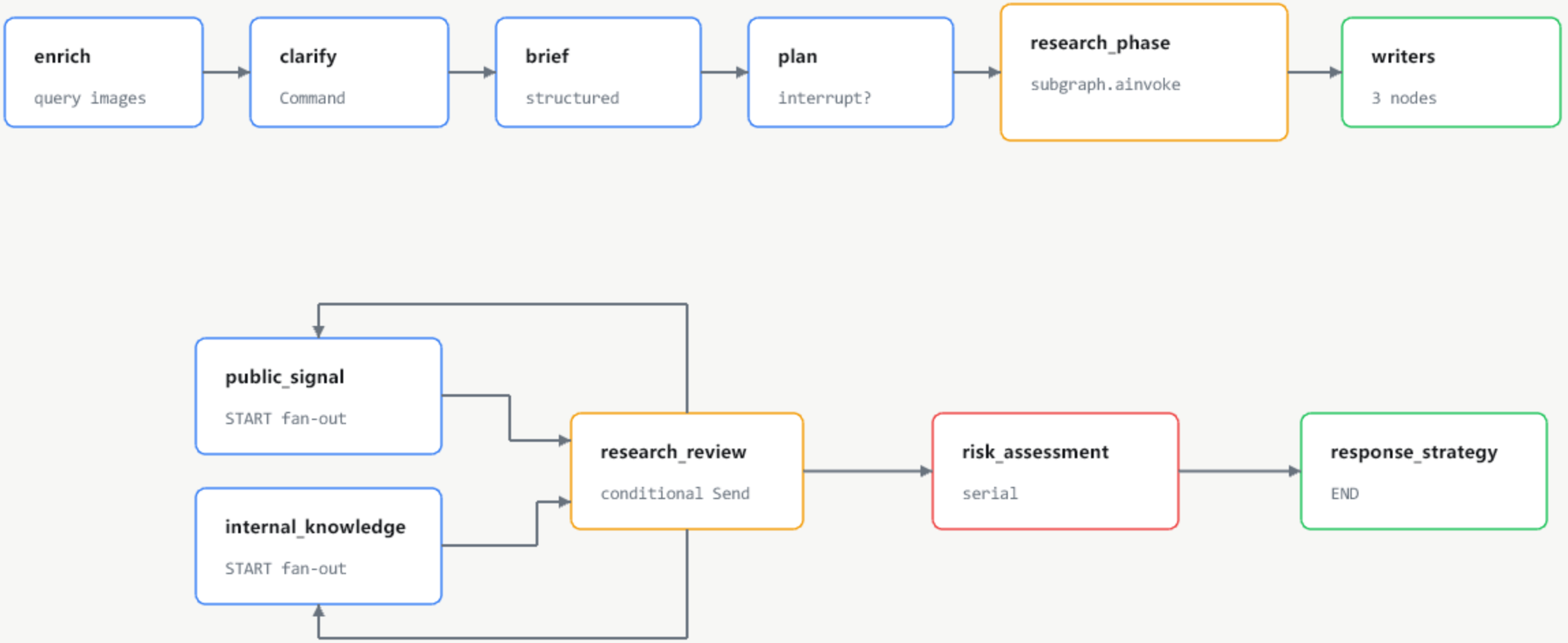
runtime

budget / metrics

累计运行数据

state.py:DeepResearchState; state.py:*_reducer

当前 LangGraph 节点与边



deep_researcher.py:1562-1640; public_opinion_builder

一次请求的完整调用链

01 ResearchRequest

02 server.research

03 graph.astream

04 write_research_brief

05 plan_report_sections

06 **research_phase**

07 run_business_agent

08 **AgentRuntime.run**

09 tool.ainvoke

10 state reducers

11 section_writer

12 compile_final_report

web/server.py; deep_researcher.py; runtime/business_agent.py; runtime/agent_runtime.py

RunnableConfig 的来源

```
config = {
  "configurable": {
    "thread_id": uuid4(),
    "research_model": request.model,
    "search_api": request.search_api,
    "rag_enabled": request.rag_enabled,
    "retrieval_mode": "hybrid" | "web_only",
    ...mode_limits,
  },
  "metadata": correlation_metadata(...),
}
```

- 调用方创建 config；LangGraph 逐节点透传
- 节点统一调用 Configuration.from_runnable_config(config)
- 模型 API key、tool 配置和预算阈值也从同一对象读取
- metadata 只承载有界关联标识，不写进 State
- Graph factory 的 config 参数被丢弃，避免双入口语义

web/server.py:research; configuration.py:Configuration.from_runnable_config

thread_id、user_id 与 research_run_id

thread_id

LangGraph conversation / resume

user_id

跨 conversation 的用户范围

research_run_id

一次研究与 Research Graph scope

来源

- Web 每次请求生成 thread_id
- Studio / API 可由调用方传入
- conversation_id 优先于 thread_id

来源

- configurable.user_id
- metadata.user_id / owner
- 可为空，不伪造

来源

- State / config / metadata 解析
- 用于图节点与 transcript 隔离
- 不同 run 不共享证据图

memory/context.py; workflow/state_utils.py:resolve_research_run_id

断点式源码追踪路线

01	web/server.py:research	看 config 和 initial_state
02	deep_researcher.py:deep_researcher	看 graph factory
03	write_research_brief	看 State Command
04	research_phase	进入子图
05	run_business_agent	组装 spec、tools、workspace
06	AgentRuntime.run	观察 model_messages / tool_calls
07	compile_final_report	看最终 report.final

langgraph.json; core source files listed below

主流程学习检查点

看完这一章，我应该能回答

01 生产环境的 Graph factory 为什么每次重新 compile?

02 State 的五个业务域如何在并行更新时避免覆盖?

03 RunnableConfig 是谁构造、由谁读取的?

04 thread_id、user_id 和 research_run_id 分别隔离什么?

05 当前生产 Graph 是否自动持久化 checkpoint?

当前 Multi-Agent 模式

结论

图编排的角色工作流

独立的部分

- 四个独立 AgentSpec
- 角色专属 system prompt
- 不同 tool domain 权限
- 私有 ReAct history / memory / summary
- 一个显式 LangGraph 子图负责交接

没有独立的部分

- 没有自由互聊的 agent swarm
- 没有每个角色一套独立子图
- 没有独立 long-term store namespace
- 没有 supervisor LLM 决定任意 handoff
- risk 与 response 不是并行执行

deep_researcher.py:public_opinion_builder;public_opinion_agents/base.py

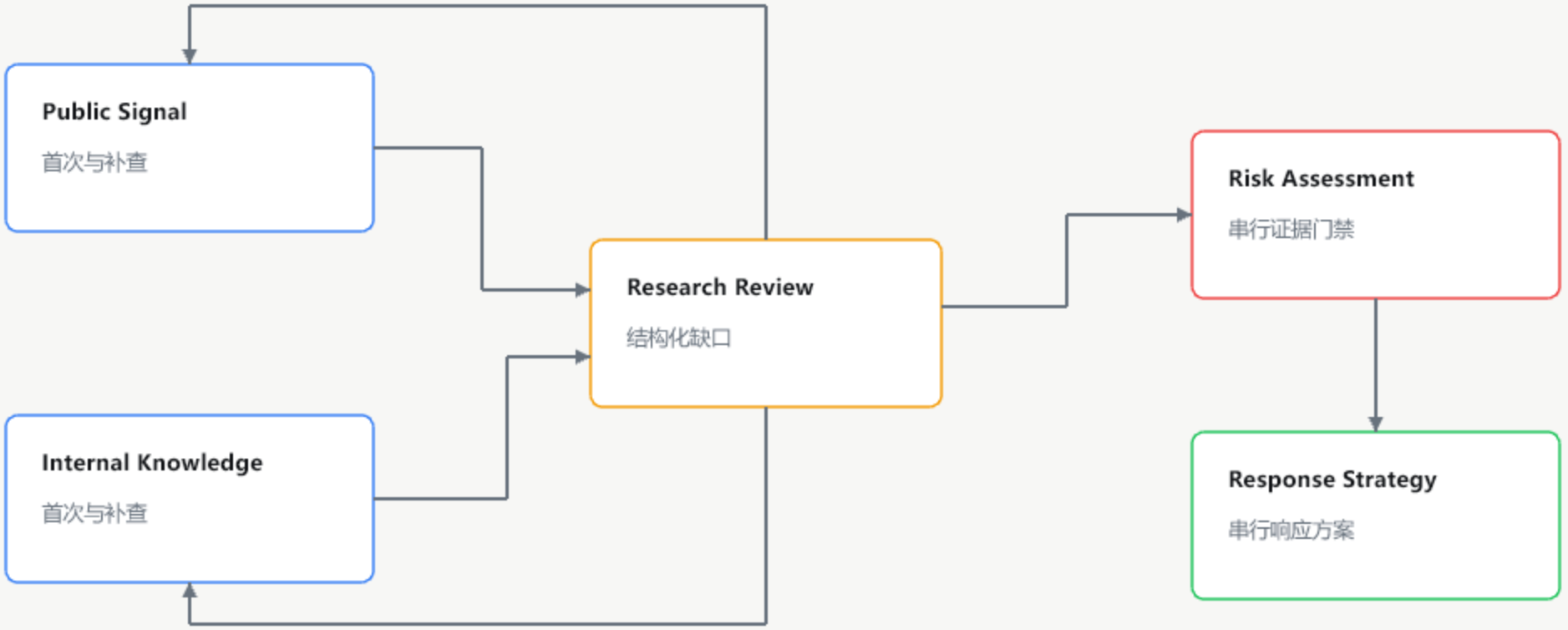
四个 Agent 的职责合同

Role	Responsibility	Allowed domains	Context strategy
public_signal	公开信号	core · web · social	producer
internal_knowledge	内部事实	core · rag	producer
risk_assessment	证据门禁与风险	core · web · rag · social	consumer
response_strategy	响应方案	core · rag	consumer

每个 AgentSpec 还包含 input_contract、output_schema、tool_policy、memory_policy、execution_strategy 和 handoff_policy。

public_opinion_agents/*.py; registry.py

Agent 协作图

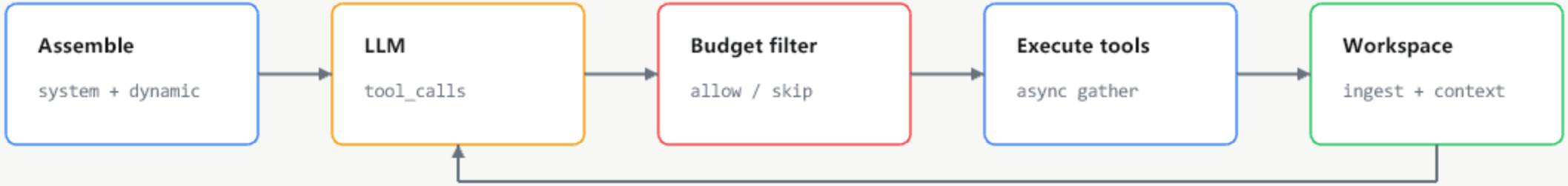


初始并行：`public_signal ‖ internal_knowledge`

补查：`Send(payload)` 只发给有 `gap` 的角色

deep_researcher.py:public_opinion_builder;route_after_research_review

共享 AgentRuntime 的 ReAct 循环



stop conditions
- no tool calls
- ResearchComplete
- max_react_tool_calls
- budget guard
- context recovery

finalization
Research Graph enabled → workspace.finalize()
otherwise → compress_report()
then → AgentRuntimeResult

runtime/agent_runtime.py:AgentRuntime.run

Agent Context 与 Memory 隔离

共享范围

- 同一 DeepResearchState schema
- 同一 RunnableConfig 和累计 budget
- 同一 research_run_id 范围
- 上游证据可通过 role reports 或 Research Graph 读取

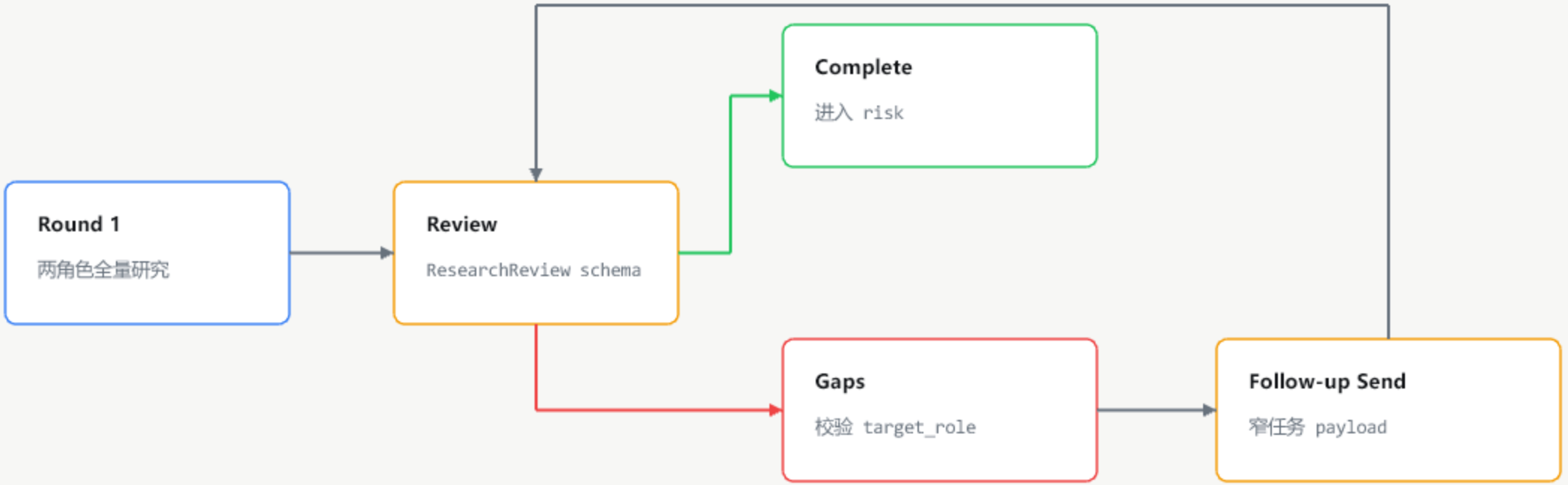
角色私有范围

- agents[role].memory 最近 5 条
- agents[role].rolling_summary
- AgentRuntime 内部 messages
- Working Context 按 role 保存
- AgentSpec 决定 prompt 与 tools

限制：生产默认没有 Checkpointer，因此跨 invoke 保留私有状态依赖外部运行时持久化。

state.py:agents_reducer; AgentRuntime.__init__; ResearchWorkspace

动态研究循环



- max_research_rounds 是安全上限，不等同预算
- 补查只允许 public_signal / internal_knowledge
- 任务去重依据 task_id 或角色 + objective + evidence
- Review 完成后 risk → response 固定串行

research_review; route_after_research_review; Send

Multi-Agent 学习检查点

看完这一章，我应该能回答

01 四个 Agent 分别独立了 Prompt、Tool、Context、Memory 中的哪些部分?

02 为什么当前实现属于 graph-based workflow，而不是 supervisor swarm?

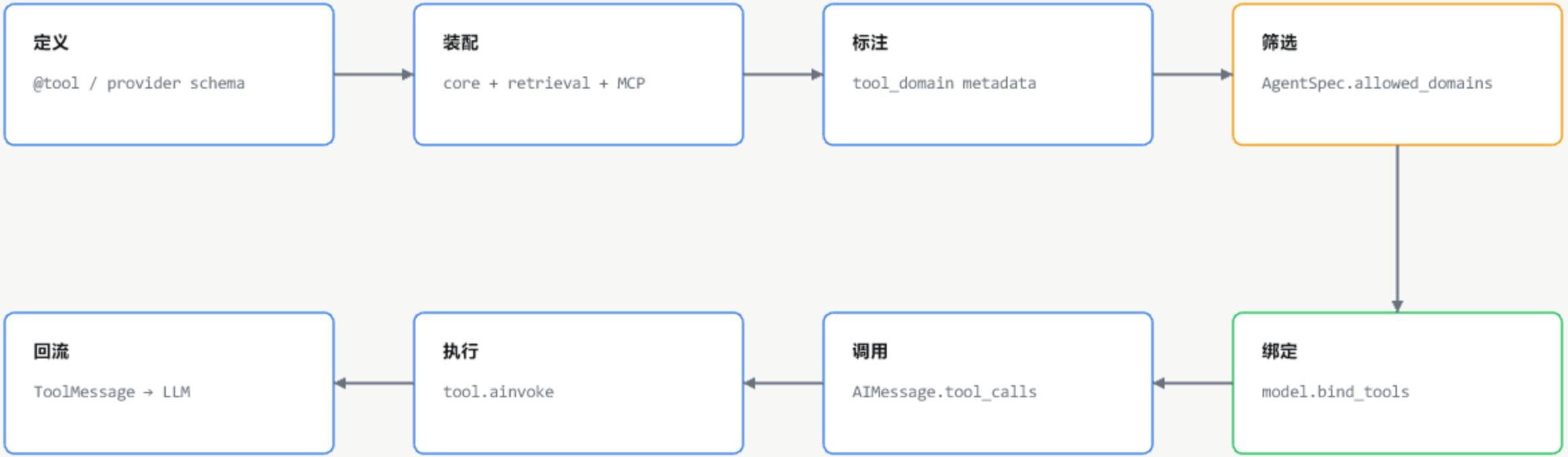
03 初始研究为什么并行，风险与响应为什么串行?

04 Research Review 怎样只把任务发给需要补查的角色?

05 并行 agent 更新怎样避免覆盖彼此的报告?

Slides 18-23; public_opinion_agents; AgentRuntime; state reducers

Tool 生命周期



Budget Guard 在执行前过滤，跳过的调用也生成合成 ToolMessage。

utils.py:get_all_tools; business_agent.py; agent_runtime.py

Tool Domain 权限矩阵

Role	core	web_search	rag	social_media	database / document / feishu
public_signal	YES	YES	-	YES	-
internal_knowledge	YES	-	YES	-	-
risk_assessment	YES	YES	YES	YES	-
response_strategy	YES	-	YES	-	-

当前四个业务角色不会获得 database、document 或 feishu domain，即使 MCP Server 已加载。

mcp/domain_filter.py; public_opinion_agents/*.py

MCP Server 到 Agent



- Legacy single-server config 可做工具白名单与认证包装
- DBHub、MarkItDown、Feishu 各自构造 transport
- 每个 Server 单独连接和发现工具
- Tool 最终仍是 LangChain BaseTool，由 AgentRuntime 执行

mcp/tools.py:load_mcp_tools; tool_wrapper.py; domain_filter.py

多 MCP 管理与冲突处理

故障隔离

```
for server_name, server_conn in servers:
    try:
        client = MultiServerMCPClient({server_name:
server_conn})
        tools = await client.get_tools()
    except Exception:
        continue
```

名称冲突

```
if original_name in existing_names:
    new_name = f"{server_name}__{original_name}"
    copy coroutine / schema / metadata
else:
    keep original name
```

- 一个 Server 失败不会阻止其他 Server
- 冲突工具不静默丢弃
- domain metadata 决定角色是否真正可见
- 当前没有全局健康检查或动态重连协调器

mcp/tools.py; mcp/tool_wrapper.py

当前 Skill 的真实边界

CURRENT FACT

当前源码树没有通用 skills/ runtime，也没有 identify_skill 或 load_skill 节点。

social_media_skill 实际是什么

- docs 中的业务组合层命名
- 实现为 social_media/tools.py 的进程内 LangChain tools
- 底层复用 social_media_mcp.py 的检索原语
- 由 public_signal / risk_assessment 的 domain 权限决定可见性

文档漂移

- README 目录树仍显示 skills/，但目录不存在
- Web SSE 过滤仍保留 identify_skill / load_skill 名称
- Git 历史没有找到主舆情 Flow 被包装为 Skill 的提交证据
- 因此 PPT 不把 Skill runtime 写成现状

rg --files; docs/guide/social-media-skill.md; README.md

Graph、Agent、Skill、MCP、Tool

Graph	**编排状态与顺序**	deep_researcher.py	业务主流程
Agent	**角色化推理合同**	AgentSpec + AgentRuntime	谁负责判断
Skill	**可复用业务方法组合**	social_media_skill 文档命名	当前无通用 runtime
MCP	**跨进程/服务能力协议**	MultiServerMCPClient	如何发现外部工具
Tool	**最小可执行动作**	BaseTool / provider schema	一次调用做什么

current source boundaries; docs/guide/social-media-skill.md

Tool、MCP、Skill 学习检查点

看完这一章，我应该能回答

01 Tool 从定义到 ToolMessage 回流经过哪些步骤?

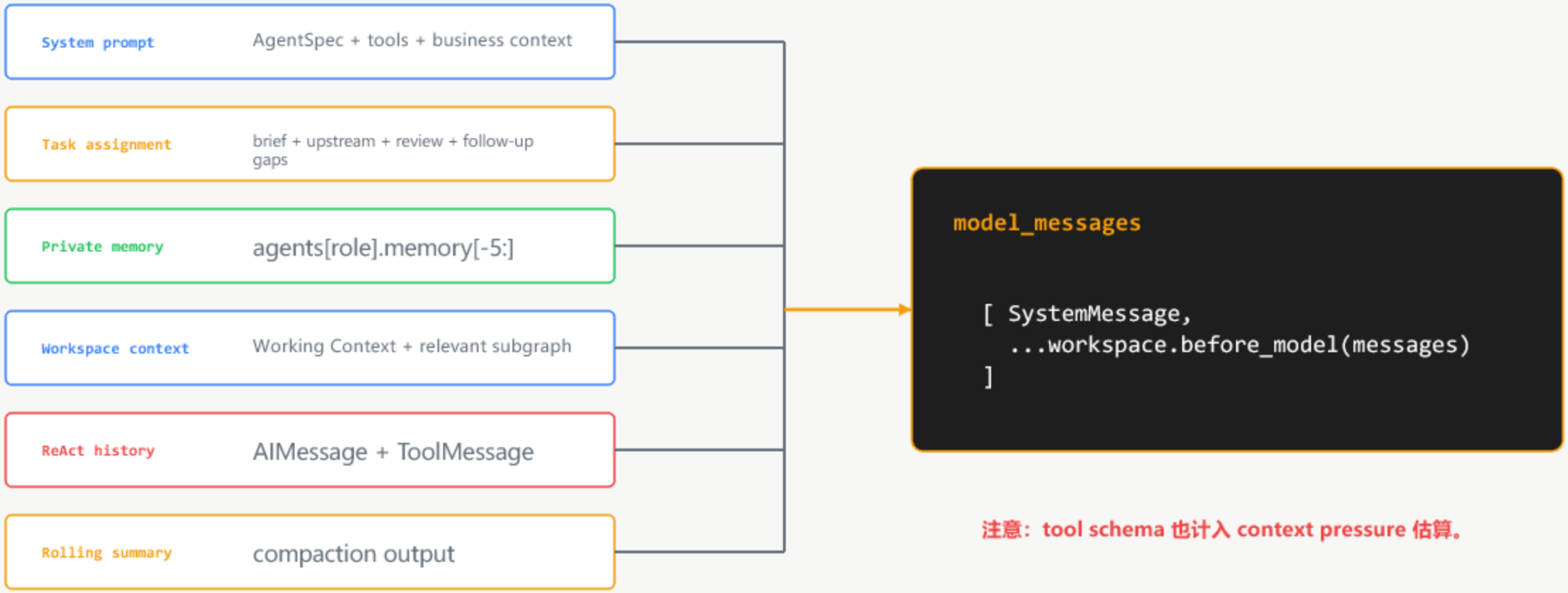
02 role domain filtering 与模型自行选工具有什么区别?

03 多 MCP Server 的故障为什么不会相互拖垮?

04 工具重名时为什么选择 server__tool，而不是静默覆盖?

05 当前项目中的 social_media_skill 与通用 Skill runtime 有何区别?

LLM Context Assembly



注意：tool schema 也计入 context pressure 估算。

AgentRuntime.run; ResearchWorkspace.before_model; AgentSpec.format_system_prompt

Memory 实现状态

本次 Graph State	IMPLEMENTED	messages + 五域 state
Agent 私有记忆	IMPLEMENTED	role memory + rolling_summary
Working Context	OPTIONAL	Research Graph enabled
MySQL 长期记忆	OPTIONAL	rag_memory_write_enabled
生产 Checkpointer	NOT WIRED	compile() 未传 checkpointer
测试 Checkpointer	IMPLEMENTED	MemorySaver in tests/evals
独立 per-agent store namespace	NOT IMPLEMENTED	隔离来自 state role keys

state.py; runtime/agent_runtime.py; memory/; tests/run_evaluate.py

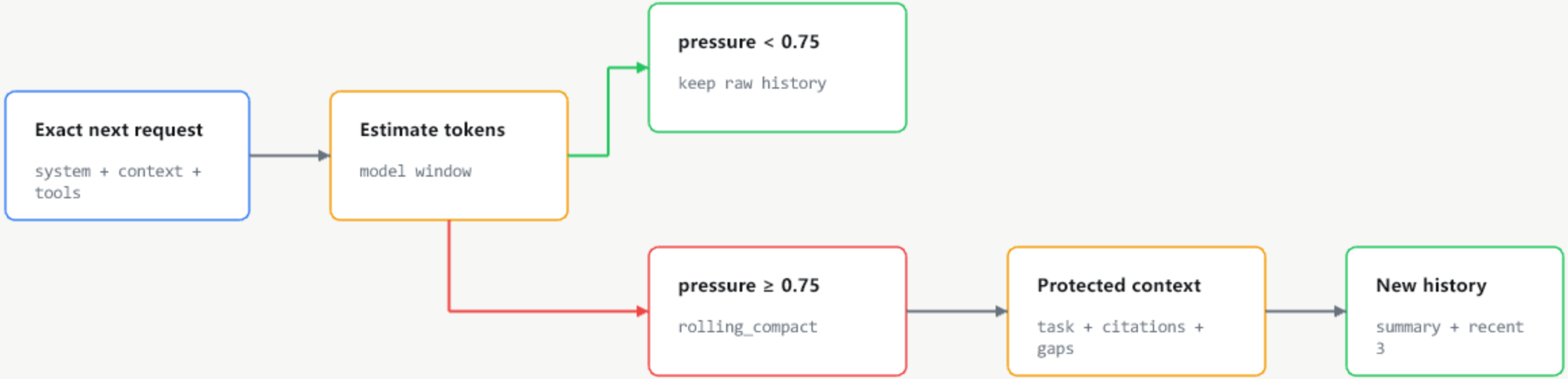
长期记忆写入与检索闭环



- conversation_id 优先，否则 thread_id，再回退 user_id / default
- user_id 与 conversation_id 同时参与过滤
- chat_raw 不进入索引，summary / facts / decision 等可索引
- 写入与后台索引失败均 fail-open，不影响报告返回

maybe_persist_chat_memory; memory/writer.py; rag/indexer.py

Context Compression



warning_ratio 默认 0.60; compaction_threshold 默认 0.75。压缩预算也计入累计 usage。

AgentRuntime._compact_history; research_graph/compaction.py:rolling_compact

Context 与 Memory 学习检查点

看完这一章，我应该能回答

01 一次模型调用最终包含哪些动态上下文?

02 Agent 私有 memory 与长期 MySQL memory 有什么不同?

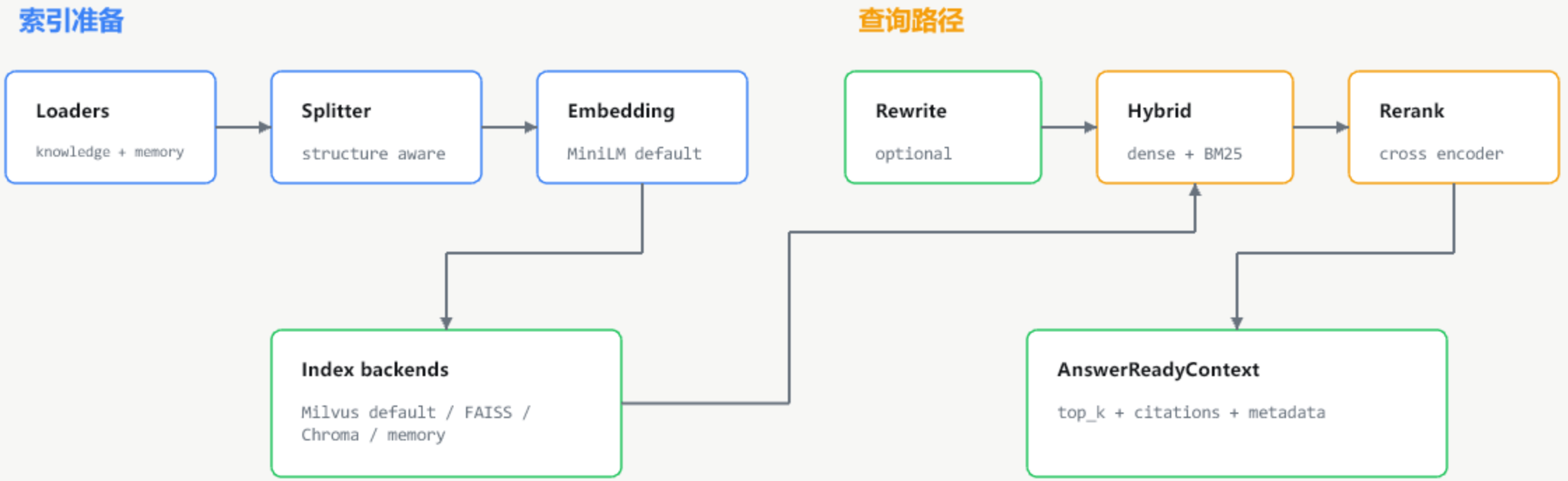
03 为什么 thread_id 不能自动等价于 user_id?

04 上下文压缩在什么阈值触发，哪些内容需要保护?

05 没有生产 Checkpointer 时，跨请求连续性依赖谁?

Slides 32-35; AgentRuntime; memory/; research_graph/compaction.py

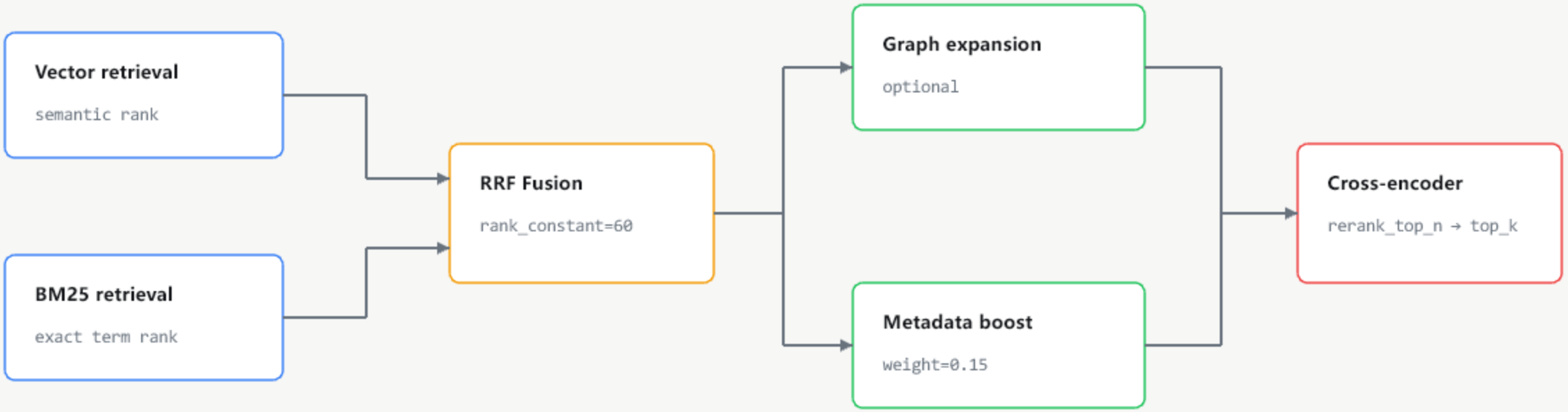
当前 RAG 架构



RAGPipeline.query() 是查询门面；RAGIndexer.ensure_ready() 负责可重建索引。

tools/rag_tool.py; rag/service.py; rag/indexer.py

Hybrid Retrieval、Fusion 与 Rerank




```
FINAL ORDER    retrieve candidates → RRF → graph / metadata → cross-encoder → authority → top_k
```

rag/retriever.py; rag/reranker.py; rag/service.py:RAGPipeline.query

GraphRAG 与 Research Graph

Knowledge GraphRAG

- 知识库索引阶段构建 chunk 关系
- 用于扩展 RAG 候选
- 可用共现、结构边和 Neo4j backend
- 结果仍进入 RRF / rerank / authority 链
- graph_enabled 默认关闭

Per-run Research Graph

- Agent 工具结果进入 run-scoped 证据图
- Producer 写入，Consumer 检索相关子图
- Working Context 有角色范围与容量上限
- 替代整份 role report 反复注入
- research_graph_enabled 默认关闭

共同点只有“图”；生命周期、数据对象、用途和配置键完全不同。

rag/graph.py; research_graph/; docs/research-graph-context-harness.md

舆情研究任务生命周期



业务分工与图顺序一致：先收集事实，再做证据门禁，最后才生成对外响应建议。

这条顺序减少“先想话术、再找证据”的确认偏误。

deep_researcher.py; four AgentSpecs; report nodes

从规划到报告

```
Planning
write_research_brief()
  → ResearchQuestion.research_brief
plan_report_sections()
  → Sections[]
  → optional interrupt feedback
```

```
Research + Writing
research_phase()
  → role reports / graph context
section_writer()
  → completed_sections
write_final_sections()
compile_final_report()
  → report.final + AIMessage
```

- 预算不足时 planner 降级为单一 research section
- section_writer 并行写需要研究的章节
- 缺失章节可由 final_report_model 补写
- 全部缺失时从 role reports 或 graph context fallback
- 最终报告可追加 token budget 摘要并可选写长期记忆

write_research_brief; plan_report_sections; section_writer; compile_final_report

Token Usage 与 Trace

LangSmith：执行追踪

- 原生 Graph / Node / Agent / LLM / Tool spans
- RAG 内部阶段使用少量 trace_span
- thread_id 等有界 metadata 做关联
- 网络与配置失败 fail-open
- 不把 trace 结果写入 State 决策

budget.py：Usage 与策略

- ModelAttemptCounter + UsageMetadata callback
- input / output / total / cache details
- 按调用累计 model、tool、search 次数
- 只信 provider reported usage，不伪造缺失 token
- 预算守卫可跳过调用并保留原因

当前不是“一个监控系统”：执行可观测与预算决策故意分离。

budget.py; observability/langsmith.py; docs/architecture/observability.md

架构演进时间线



git log origin/main; selected commits in sources.md

四次关键重构

旧问题	调整	收益与代价	Commit
通用模式分叉	舆情主流程收敛	少一套 supervisor/legacy 路径	aff29ea
一次研究难补缺口	Review + 定向 Send	质量提高，但 follow-up 成本上升	5989480
State 与角色运行重复	五域 State + 共享 AgentRuntime	并行合并与职责更清晰	3f0a548 / 5cc29c0
Observer sidecar 与 LangSmith 重叠	LangSmith + budget.py	更少重复埋点，责任边界清楚	d18c87a

aff29ea; 5989480; 3f0a548 + 5cc29c0; d18c87a

源码学习地图

Route 1

LangGraph

langgraph.json
deep_researcher.py
state.py
workflow/state_utils.py

逐文件打断点

Route 2

Multi-Agent

public_opinion_agents/
runtime/business_agent.py
runtime/agent_runtime.py
research_graph/workspace.py

逐文件打断点

Route 3

MCP / Tools

utils.py
mcp/tools.py
mcp/tool_wrapper.py
mcp/domain_filter.py

逐文件打断点

Route 4

RAG / Memory

tools/rag_tool.py
rag/service.py
rag/retriever.py
memory/writer.py

逐文件打断点

建议每条路线先跑对应测试，再进入可选外部基础设施。

最终学习检查点

看完这一章，我应该能回答

01 我能从入口一路跟到最终 report.final，并说出每个 State patch 的来源吗?

02 我能解释四个 Agent 独立了什么、共享了什么、谁决定下一步吗?

03 我能区分 Tool、MCP、Skill、Agent、Graph 的责任边界吗?

04 我能区分 GraphRAG、Research Graph、Agent Memory 与 MySQL Memory 吗?

05 我能从 Git 提交说明一次架构调整的动机、收益和代价吗?